CS229 Notes: Regularization & Model Selection

Section 1: Cross-Validation

- **Core Problem:** Several models → which one to pick?
- **(X) Wrong instinct:** pick the one with lowest training error.
  Why wrong? → A 10-degree polynomial will memorize 10 points perfectly.
  → Training error = 0
  but, fails on new data.
  → high variance / overfitting.

Actually we should choose a model based on generalization error which is during prediction using testing.

Soln → use →

(i)

- **Hold-Out Cross Validation:**
  (split data →) 70% → training (S-train)
  30% → test (S-cv) (model never sees this during training)

Ex → 1000 house price examples
- M1: linear fit (degree 1)
- M2: quadratic (2nd degree)
- M5: 5 degree

choose 3 (ω →)

Train all three on 700 examples → Test error on 300 held-out:
- M1: 15% error
- M2: 9% error
- M5: 22% error (overfitting kicks in)

→ picking M2
(but are wasted 300 examples.)

(ii) Soln →

- **K-Fold Cross Validation:**

Soln → reuse the data smarter.
→ split into K = 10 equal chunks → For each chunk j:
- Train on the other 9 chunks
- Test on chunk j → could be 7th/8th/5th/1st
- get error → on j

Ex → (concrete example → K = 3, 9) →
Fold 1: train on [4,5,6,7,8,9], test on → [1,2,3] → error = 12%.
Fold 2: train → [1,2,3,7,8,9], test on → [4,5,6] → error = 10%.
Fold 3: train → [1,2,3,4,5,6], test → [7,8,9] → error = 14%
Estimated generalization error = (12+10+14)/3 = 12%

(viii) Leave-One-Out (CV) → Extreme version

- train n-1 ex, test on 1 ex → repeat m times
  - → more data usage
  - → more compute cost

(9) How are we going to identify which model is sensitive to small data changes and which are to large data changes → without doing experiment testing (costly)?

⇒ Model known to be sensitive for small data changes ⇒

(1) Neural Networks – different random initializ. + slightly different data = very different weights

(2) Decision trees → 1 diff point near root → diff tree structure

(3) Boosting methods → early weak learner affect everything downstream

- Models that are relatively stable ⇒

(1) Linear / Logistic Regression – convex loss, unique sol^n, small data changes = small parameter changes.

(2) Naive Bayes – counting probabilities, very stable.

(3) SVM with large margin – support vectors dominate, most points don't even matter.

Rule :– If model has a non-convex loss or involves sequentially / greedy decisions → be cautious about retraining → sensitive

If it has a convex loss with unique sol^n → retrain freely → non sensitive

Sec-3 : Bayesian Statistics & Regularization

in M.L.E → "what θ makes the data most likely?"

$$\theta_{MLE} = \arg\max_{\theta} \prod_{i=1}^{m} P(y^{(i)} \mid x^{(i)} ; \theta)$$

Assumption → θ is fixed unknown constant, it exist, we just don't know it.

The Bayesian shift → says → θ is not a fixed constant. It is a random variable with its own distribution. Before seeing any data, we have prior belief about θ — written $p(\theta)$.

→ Analogy ⇒ Guessing Std exam score = θ.
without prior, scores → 40–90, clustered around 70 (Gaussian)
with prior, homework grades (data) → updating belief
→ updated belief = posterior

- The posterior after seeing training data S:

$$p(\phi|S) = \frac{p(S|\phi) \cdot p(\phi)}{p(S)}$$

$p(\phi|S)$ → posterior; $p(S|\phi)$ → likelihood; $p(\phi)$ → prior; $p(S)$ → normalizing constant (can be ignored)

posterior ∝ likelihood × prior

- Fully Bayesian prediction ⇒

To pred on x (new data) → avg over all possible θ, weighted by → how probable each θ is:

$$p(y|x,S) = \int_{\theta} p(y|x,\theta)\, p(\theta|S)\, d\theta$$

$p(y|x,\theta)$ → all possible; $p(\theta|S)$ → how probable each θ is

- Intuition ⇒ don't commit to 1 θ, consider all θ values, plausible (make sense)

problem ⇒ Integral is impossible to compute in closed form as θ is high dimensional — imagine integrating over 10,000 dims.

- MAP :- The practical Approximation.
Instead of integrating over all θ, just find the single most probable θ under the posterior ⇒

$$\theta_{MAP} = \arg\max_{\theta} \prod_{i=1}^{m} p(y^{(i)}|x^{(i)},\theta) \cdot p(\theta)$$

Compare with MLE ⇒

$$\theta_{ML} = \arg\max_{\theta} \prod_{i=1}^{m} p(y^{(i)}|x^{(i)};\theta)$$

→ Diff is MAP has extra $p(\theta)$ → probable values of θ

- closed form v/s other sol'ns
  - closed form → exact, compact, satisfying formula (π)
  - other sol'ns → numerical approx (3.14159)
- closed form v/s open form
  - closed form → finite ops ($+, -, *, /$, exp, root) → no iterations or loops (Gradient descent) → functions used (polynomial, exponential, trigonometric etc)
  - open form → infinite no of operations → infinite series, continued fractions, integrals

Now Connection to Regularization :-

what if your prior is $\Rightarrow \theta \sim N(0, \sigma^2 I)$

"I believe θ is probably small — centered at zero"

- Taking log of the MAP objective (log turns products → sums)

$$\theta_{MAP} = \arg\max_{\theta} \sum_{i=1}^{m} \log p(y^{(i)} | x^{(i)}, \theta) + \log P(\theta)$$

- Plug in the Gaussian prior — log of Gaussian is a quadratic:

$$\log p(\theta) \propto -\frac{1}{2\sigma^2} \|\theta\|^2$$

- So MAP becomes:

$$\theta_{MAP} = \arg\max_{\theta} \sum_{i=1}^{m} \log p(y^{(i)} | x^{(i)}, \theta) - \frac{1}{2\sigma^2} \|\theta\|^2$$

- Equivalently (flipping sign, max → min):

$$\theta_{MAP} = \arg\min_{\theta} \underbrace{\sum \text{loss}}_{\text{fit the data}} + \underbrace{\lambda \|\theta\|^2}_{\text{keep } \theta \text{ small}}$$

we found → L2 regularization (Ridge Regression)

- The Great Intuition:-

MLE: "Find θ that explains the data best. I don't care how large θ gets."

MAP with Gaussian prior: "Find θ that explains the data well, BUT I have prior belief θ should be small. Don't go crazy with large weights."

- Large weights = model is very sensitive to i/p changes = overfitting that.
- the prior penalizes $\lambda \|\theta\|^2$.

This is what Regularization is.

- Why MAP and Regularization reduces overfitting?

→ 2nd of prior term → $P(\theta)$ in PMLE
→ $\lambda|\theta|^2$ ↔ similar in Regularization